// 2026.03.30

[PROOF OF CONCEPT]

AUTONOMOUS SIEM SERVER & ENDPOINT EDR FRAMEWORK

BLACKBOX

SIEM Telemetry Ingestion // Endpoint Detection // Active Response

BASH | SOCAT | INOTIFY | JQ ENGINE | IPTABLES | BLUE TEAM

Denis Albin

albindeniswork@gmail.com

Cybersecurity Student | SOC & Blue Team

01 // Executive Summary

BlackBox is a two-tier Linux security information and event management **(SIEM)** and endpoint detection and response **(EDR)** platform written from scratch in pure Bash. It provides an end-to-end defensive operations workflow: from real-time kernel filesystem, process, socket, and session telemetry collection on Linux endpoints, to centralized TCP stream ingestion, boolean event querying, MITRE ATT&CK rule correlation, automated process termination, and dynamic **iptables** host firewall isolation.

KEY CAPABILITIES

- Non-blocking TCP socket ingestion (socat TCP:9000) with jq JSON schema validation and alert routing
- Interactive ANSI SOC console featuring host metrics, daemon health, and a 15-tick volume trend sparkline
- Boolean search engine (AND / OR / NOT) and recursive jq paths(scalars) JSON field hierarchy profiler
- Real-time inotifywait filesystem integrity sensor with TTL-cached SHA-256 binary hashing
- Continuous /proc process tree inspection, ss -antupH socket radar, and who user login tracking
- MITRE ATT&CK process detection engine with per-PID deduplication and 10-second rule cooldowns
- Autonomous active remediation: SIGTERM to SIGKILL process termination and BLACKBOX iptables chains
- One-line HTTP :9100 endpoint systemd enrollment and 60-second zero-downtime policy hot-reload

TOOL STACK

`Bash` `socat` `jq` `inotifywait` `iptables` `sha256sum` `ss / procfs` `openssl`

WORKFLOW



02 // Setup & Daemon Bootstrap

The **SIEM.sh** entry point begins by verifying root privileges and checking for required core dependencies **(socat, jq,** and **openssl),** offering automated package installation across **apt, yum,** and **pacman** distributions. On first launch, it invokes **core/setup.sh** to configure ingestion and web distribution ports with live socket conflict checks via **ss -ltn.**

```
==================================
     BLACK BOX SIEM SETUP
==================================
Enter SIEM Ingest port (Logs) [default: 9000]: 9000
Enter SIEM Web port (Files) [default: 9100]: 9100
Log directory name [default: logs]: logs
Service PIDs directory [default: services-logs]: services-logs
Enable EDR Active Response (Kill on Match)? (y/n) [default: y]: y
Enable SIEM-managed Firewall? (y/n) [default: n]: y
[+] Saving system configuration...  [+] Initializing data structures...
Agent Enrollment URL: http://192.168.58.10:9100
```

Fig 1. Interactive SIEM setup wizard with live port conflict validation

BACKGROUND DAEMON SUPERVISION

Once initialized, **SIEM.sh** provisions the JSON storage schemas **(agents.json, rules.json, policies.json,** and **firewall.json),** launches the three non-blocking background workers **(ingest.sh, parser.sh,** and **web-server.sh),** tracks their PIDs under **services-logs/,** and packages the EDR agent archive via **core/prep-deployment.sh.**

```
root@soc-server:~/blackbox-siem-edr/server (SIEM)# ./SIEM.sh
================================
   Black Box SIEM Framework
================================
Version: 1.0
[SIEM] Starting missing services...
[SIEM] Starting ingestion engine...   [PID: 44102 · socat TCP:9000]
[SIEM] Starting parser engine...      [PID: 44108 · jq stream router]
[SIEM] Starting WEB server...         [PID: 44115 · HTTP :9100]
[SIEM] Preparing EDR deployment packages...  [*] Packaging EDR agent...
[SIEM] System ready. Launching dashboard...
```

Fig 2. Background daemon bootstrap and automated EDR package creation

03 // SOC Console & Sparkline

The **cli/dashboard.sh** module provides a flicker-free ANSI terminal operator console using cursor-home redrawing. The main view displays live host telemetry (uptime, **/proc/loadavg,** RAM, disk usage, and RX/TX interface byte counters from **/proc/net/dev),** real-time daemon PID health, and rapid dispatch into six operational sub-consoles.

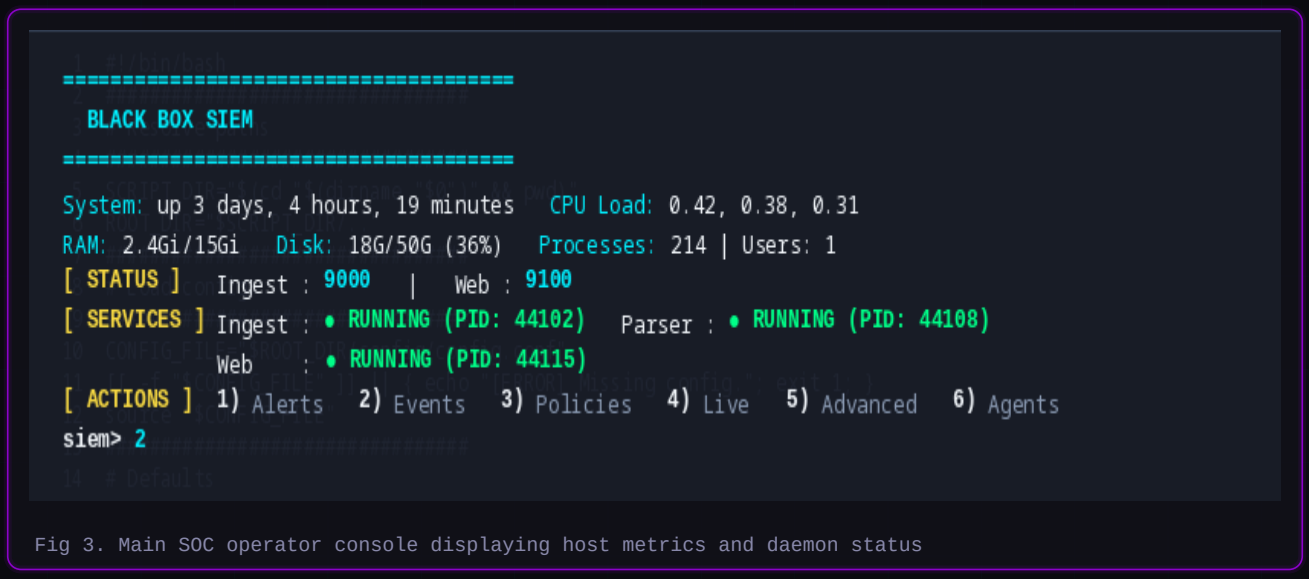


Fig 3. Main SOC operator console displaying host metrics and daemon status

REAL-TIME EVENT STREAM & SPARKLINE TREND

Selecting **2) Events** launches the real-time event stream monitor. Every 0.8 seconds, the console calculates incoming log velocity deltas and renders a 15-tick Unicode block sparkline (8-level bars) that shifts color from green to red during high-volume bursts, alongside a 10-row formatted preview table.

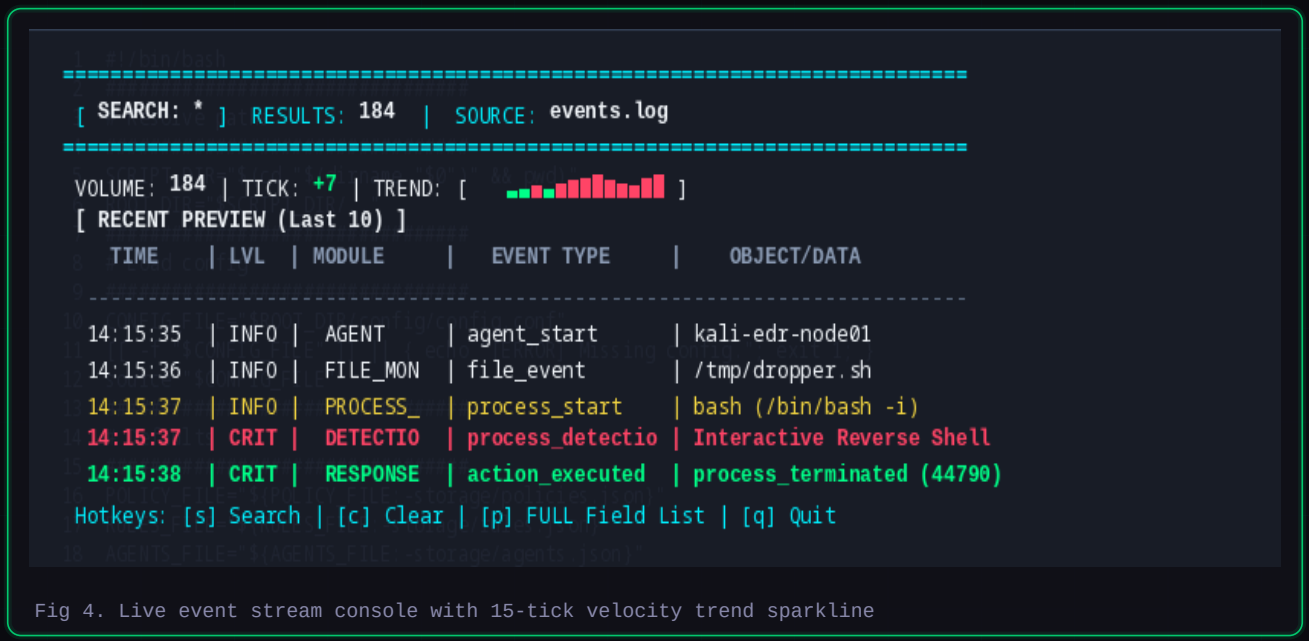


Fig 4. Live event stream console with 15-tick velocity trend sparkline

04 // Query Engine & Field Profiler

To hunt threats across high-volume telemetry logs, **engine/query.sh** compiles human-readable search expressions into dynamically constructed **jq select(...)** AST filters. It supports chained **AND, OR,** and **NOT** boolean operators, key-value matching across both top-level and nested . **data.<field>** paths, wildcard existence checks **(field=*),** and case-insensitive regex.

```
Query > level=CRITICAL AND module=DETECTION NOT host=workstation-04
{
  "timestamp": "2026-03-30T14:15:37+02:00",
  "host": "kali-edr-node01",
  "level": "CRITICAL",
  "module": "DETECTION",
  "event_type": "process_detection",
  "data": {
    "rule_id": "Interactive Reverse Shell",
    "mitre_technique": "T1059.004",
    "process_name": "bash",  "severity": "CRITICAL",  "pid": 44790
  },  "processed_at": "2026-03-30T12:15:37Z"
}
```

Fig 5. Boolean query execution filtering CRITICAL MITRE ATT&CK detections

RECURSIVE JSON SCHEMA & CARDINALITY DISCOVERY

Pressing **[p] FULL Field List** inside the event console pipes the active dataset through **jq -r 'paths(scalars) | join(".")' | sort | uniq -c | sort -nr,** automatically discovering every nested JSON key hierarchy and color-coding field frequency counts across the entire log store.

```
FIELD DISCOVERY: GLOBAL LOGS
+=================================================================+=======+
|  FIELD PATH (JSON Key Hierarchy)                                | COUNT |
+=================================================================+=======+
|  timestamp ..................................................... |   184 |
|  host .......................................................... |   184 |
|  data.pid ...................................................... |   142 |
|  data.command_line ............................................. |   118 |
|  data.sha256 ................................................... |    24 |
|  data.mitre_technique .......................................... |     6 |
+=================================================================+=======+
```

Fig 6. Automated JSON field path discovery and cardinality table

05 // Fleet HUD & Agent Enrollment

As endpoints stream events to **TCP:9000, core/parser.sh** extracts host metadata and invokes **core/agenttracker.sh** to maintain **storage/agents.json.** Each new host is assigned a unique cryptographic ID via **openssl rand -hex 4,** while **heartbeat** beacons update CPU load and RAM consumption. The **Live Telemetry HUD** flags any endpoint silent for over 45 seconds as **OFFLINE.**

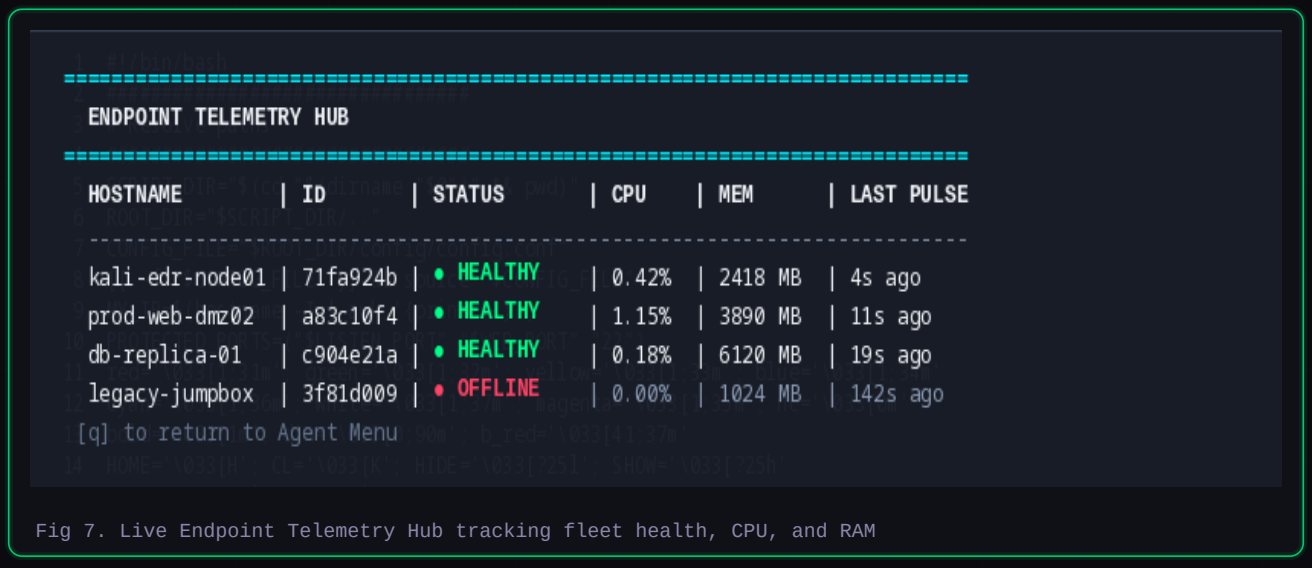


Fig 7. Live Endpoint Telemetry Hub tracking fleet health, CPU, and RAM

ZERO-TOUCH HTTP :9100 SYSTEMD DEPLOYMENT

Selecting **Enroll New Endpoint** runs **core/prep-deployment.sh,** writing the SIEM connection coordinates to **connection.info** and archiving the agent tree into **agent_package.tar.gz.** Target Linux hosts enroll with a single **curl | sudo bash** command that installs **/opt/blackbox-edr,** locks permissions to **chmod 700,** and enables the **blackbox-edr.service** systemd unit.

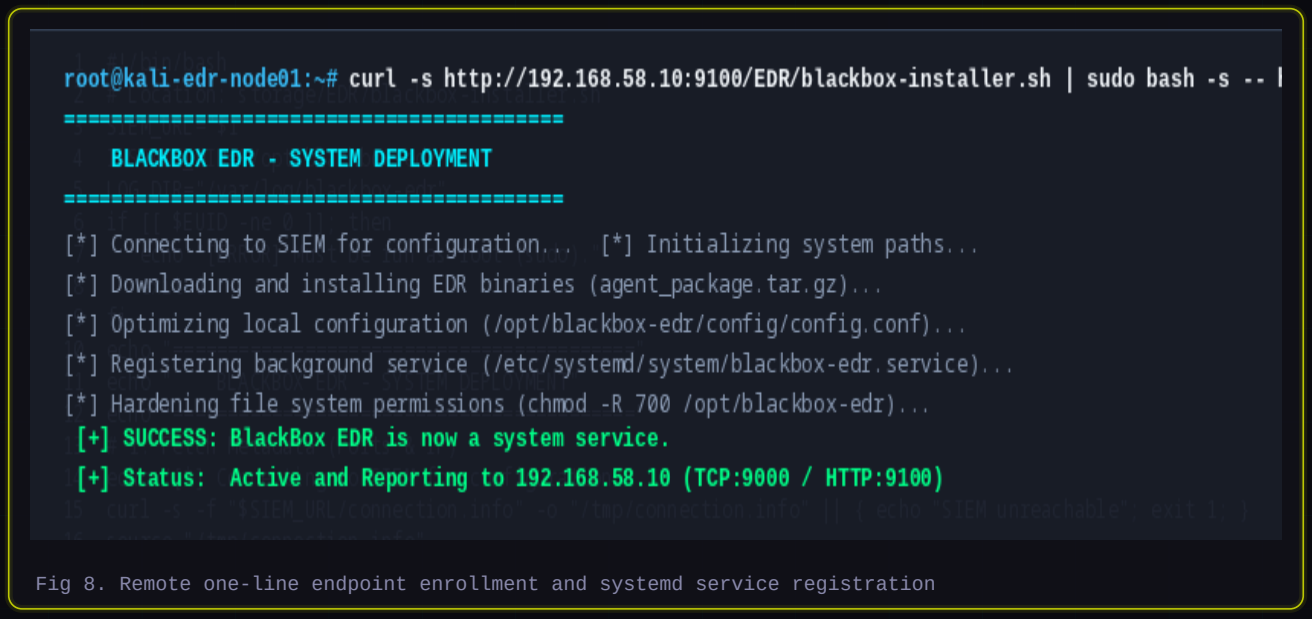


Fig 8. Remote one-line endpoint enrollment and systemd service registration

06 // EDR Telemetry & Sensor Matrix

On every enrolled workload, **agent/agent.sh** runs four concurrent telemetry collectors. The **file_monitor** worker attaches kernel **inotifywait** watches **(create,delete,move,close_write)** across sensitive directories **(/tmp, /home, /var/www, /opt)** and computes **SHA-256** hashes on executable extensions with a 300-second TTL cache to prevent disk I/O storms.

```
# Real-time file_event captured by inotifywait -m -r -e create,delete,move,close_write
{
  "timestamp": "2026-03-30T14:15:36+02:00",
  "host": "kali-edr-node01",  "level": "INFO",  "module": "FILE_MONITOR",
  "event_type": "file_event",
  "data": {
    "file_path": "/tmp/dropper.sh",
    "sha256": "5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03",
    "action": "CREATE",  "owner": "root",  "permissions": "755"
  }
}
```

Fig 9. Real-time inotifywait file creation alert with SHA-256 hash

PROCFS PROCESS TREE, SOCKET RADAR & LOGIN SESSIONS

Every 2 seconds, **process_monitor** sweeps **/proc/<pid>/stat, /proc/<pid>/exe,** and **/proc/<pid>/status** to capture newly spawned processes, parent binaries, UIDs/GIDs, working directories, and full command lines. Concurrently, **network_monitor** polls **ss -antupH** to map active TCP/UDP sockets to PIDs, while **login_monitor** tracks interactive TTY sessions.

```
# /proc/<pid> process_start + ss -antupH network_connection + who user_login
{"module":"PROCESS_MONITOR","event_type":"process_start","data":{
  "pid":44790, "ppid":44120, "user":"www-data", "uid":33, "process_name":"bash",
  "parent_process":"apache2", "process_path":"/usr/bin/bash",
  "command_line":"/bin/bash -i"}}
{"module":"NETWORK_MONITOR","event_type":"network_connection","data":{
  "pid":44790, "process_name":"bash", "protocol":"tcp",
  "source_ip":"192.168.58.142", "destination_ip":"10.10.14.99", "destination_port":4444,
  "connection_state":"ESTAB"}}
```

Fig 10. Correlated process_start and outbound C2 network_connection telemetry

07 // ATT&CK Rules & Active Kill

SOC operators author detection rules directly inside **cli/agent-mgmt.sh** (`Option 3: EDR Rules`), defining exact **process_name** matches or substring **command_line** signatures mapped to **MITRE ATT&CK** technique IDs and severity tiers **(LOW, MEDIUM, HIGH, CRITICAL).** Rules are stored in **storage/rules.json** and distributed automatically to all active endpoints.

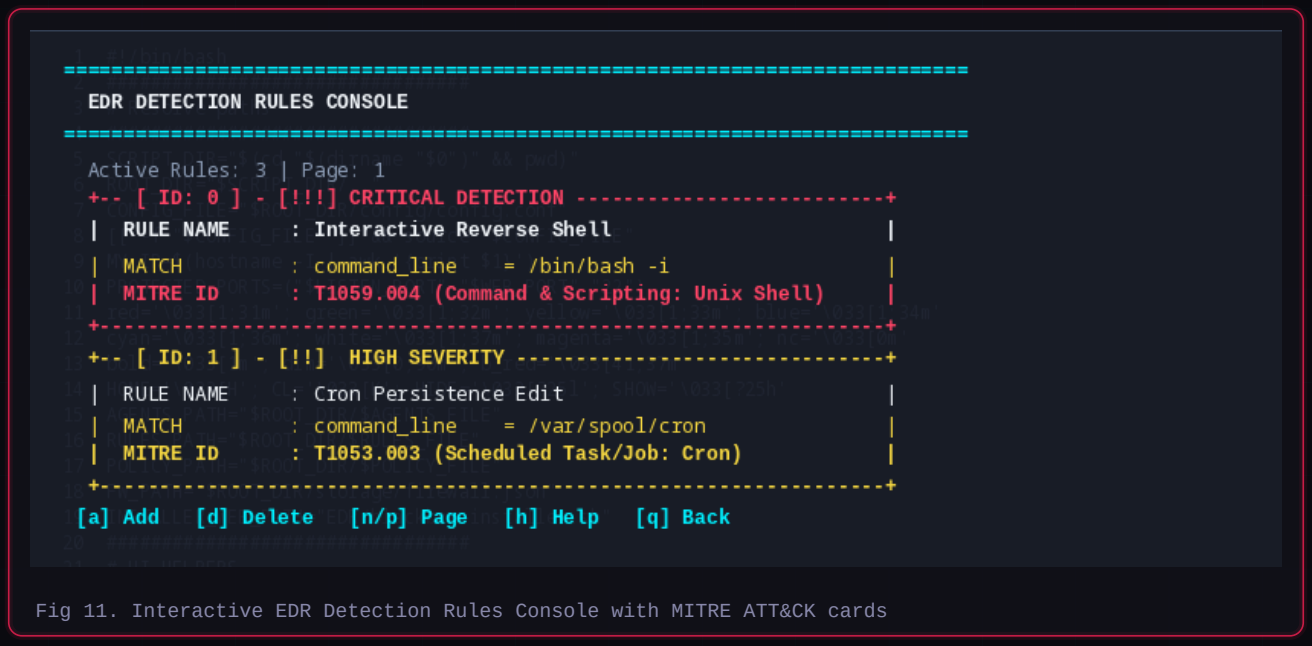


Fig 11. Interactive EDR Detection Rules Console with MITRE ATT&CK cards

AUTONOMOUS SIGTERM TO SIGKILL PROCESS REMEDIATION

On the endpoint, **core/detection-engine.sh** evaluates every **process_start** event against **rules.json.** When a **CRITICAL** rule matches and **edr_response** is enabled, it invokes **core/response.sh PROCESS <pid>,** which verifies PID liveness **(kill -0),** sends **SIGTERM,** waits 1 second, escalates to **SIGKILL (kill -9)** if needed, and streams **action_executed** to the SIEM.

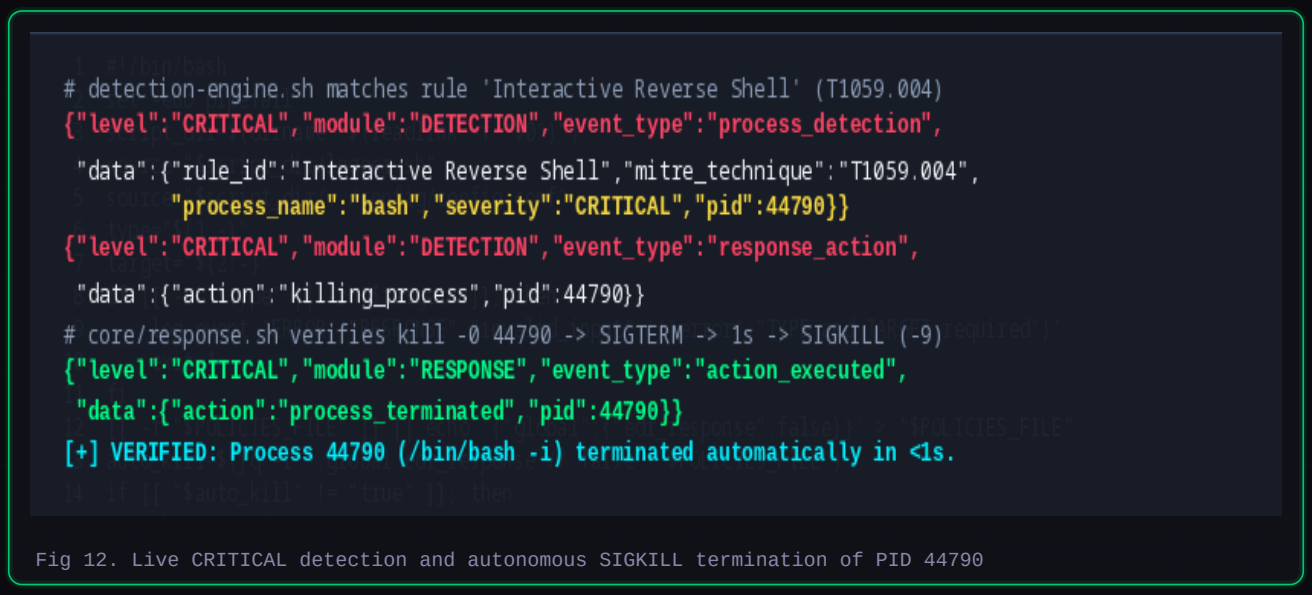


Fig 12. Live CRITICAL detection and autonomous SIGKILL termination of PID 44790

08 // Firewall & Lockout Safeguards

The **Network Firewall Console** in **cli/agent-mgmt.sh** allows operators to push centralized inbound and outbound IP/port **DROP** rules across the fleet. To prevent accidental administrative lockout, **validate_ip** and **validate_port** strictly reject rules targeting the SIEM server IP, loopback **(127.0.0.1),** SSH **(22),** or the SIEM ingestion and web ports **(9000, 9100).**

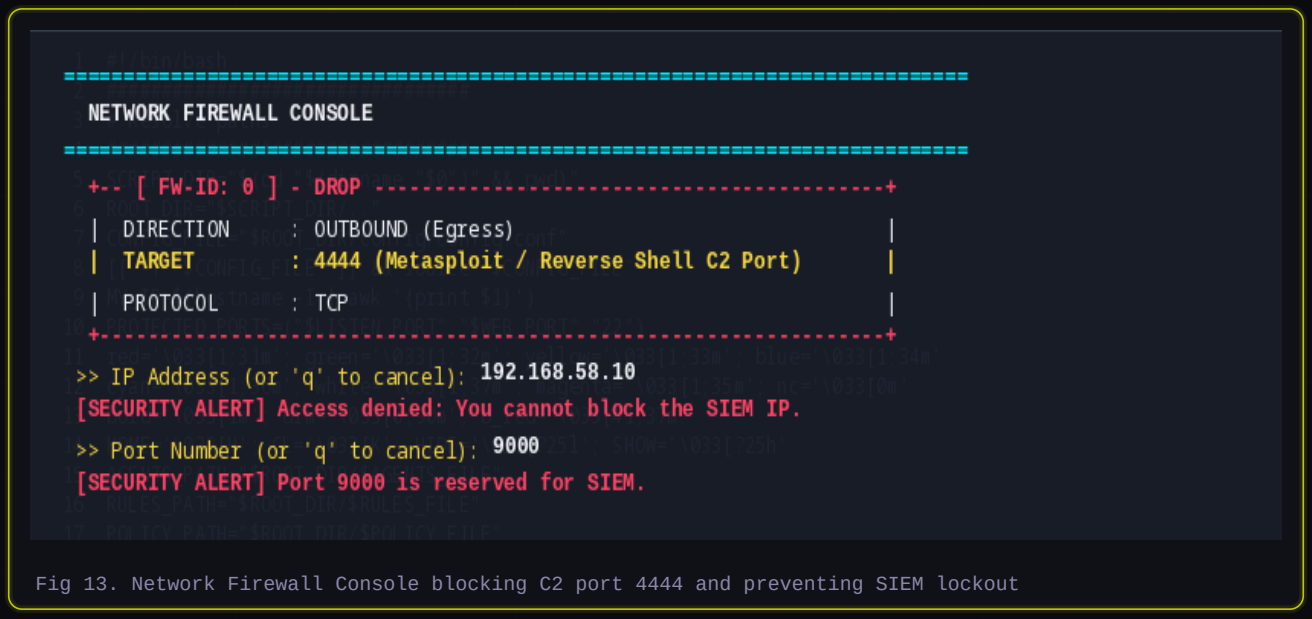


Fig 13. Network Firewall Console blocking C2 port 4444 and preventing SIEM lockout

DEDICATED BLACKBOX-IN & BLACKBOX-OUT IPTABLES CHAINS

On each endpoint, **core/firewall.sh** provisions isolated **BLACKBOX-IN** and **BLACKBOX-OUT iptables** chains jumped from position 1 of **INPUT** and **OUTPUT.** It pre-populates stateful **ESTABLISHED,RELATED** rules and explicit **ACCEPT** rules for SSH and SIEM ports before compiling each JSON rule from **firewall.json** into kernel netfilter rules.

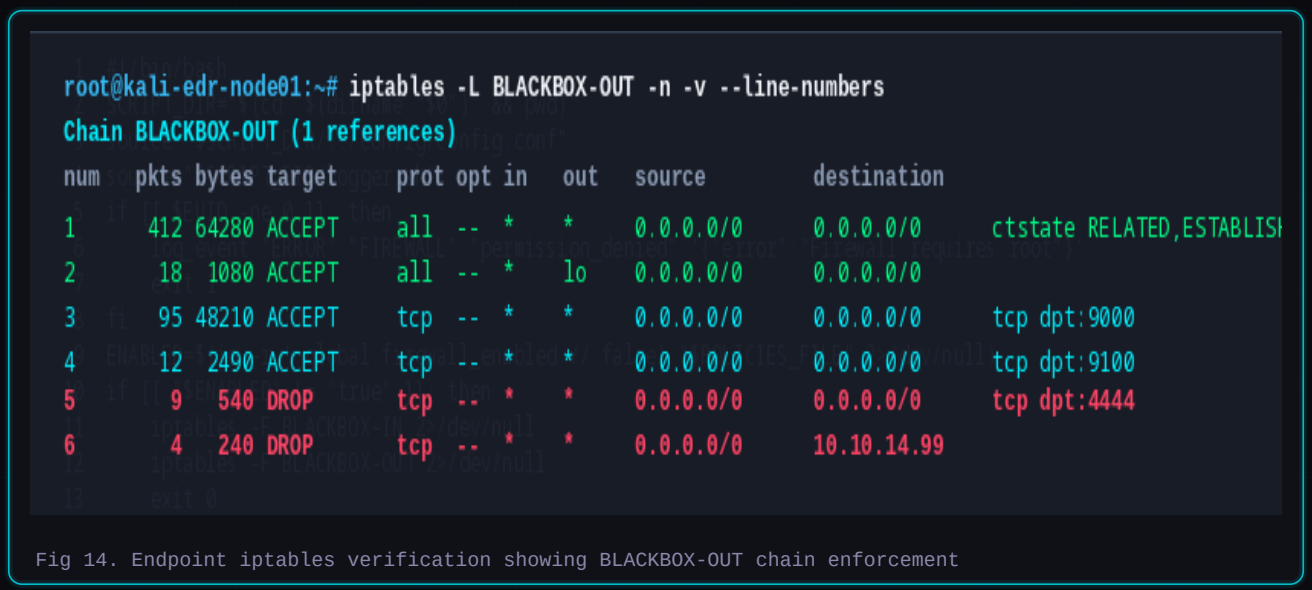


Fig 14. Endpoint iptables verification showing BLACKBOX-OUT chain enforcement

09 // Engine Tuning & Policy Sync

To balance detection sensitivity against endpoint CPU and storage overhead, **cli/agent-mgmt.sh** (`Option 5: Engine Params`) provides an interactive editor for **storage/policies.json.** Operators can append, remove, or overwrite watched directory paths, executable hash extensions, maximum file size thresholds, scan intervals, and per-sensor deduplication TTL windows.

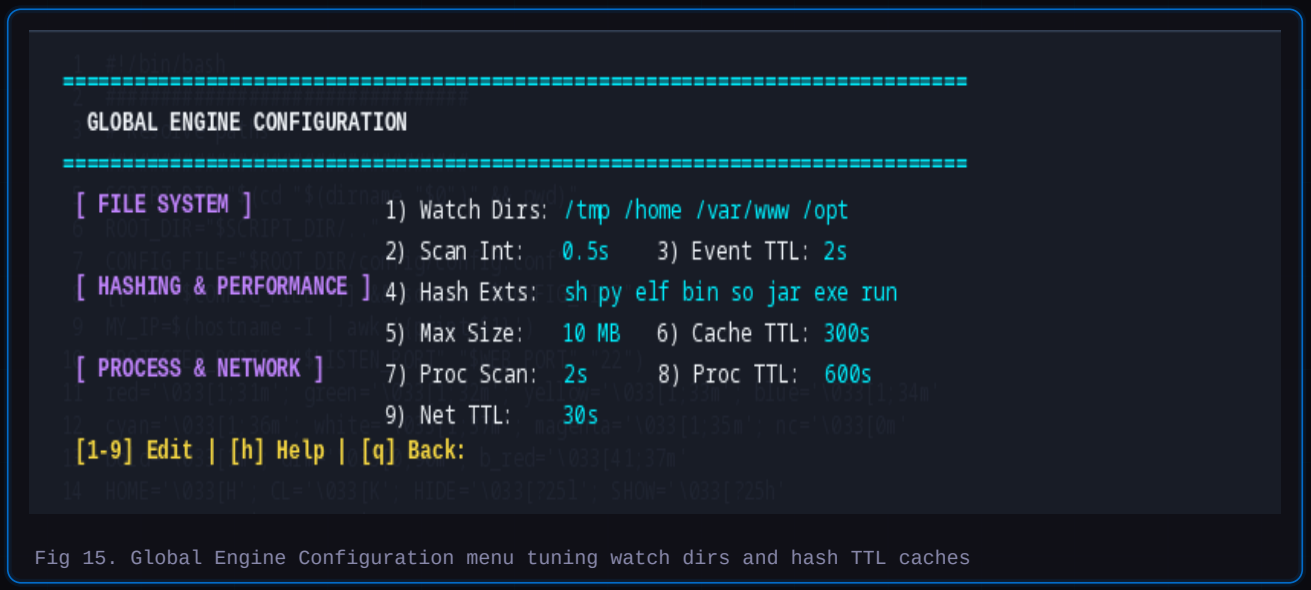


Fig 15. Global Engine Configuration menu tuning watch dirs and hash TTL caches

AUTONOMOUS 60-SECOND POLICY SYNCHRONIZATION LOOP

Every 60 seconds, **core/sync.sh** on each endpoint fetches **policies.json, rules.json,** and **firewall.json** from the SIEM HTTP distribution server **(:9100).** It validates syntax via **jq,** diffs against local configs, hot-reloads **core/firewall.sh** when firewall rules change, and gracefully cycles the sensor and detection workers via **Main.sh** when rules or TTLs update.

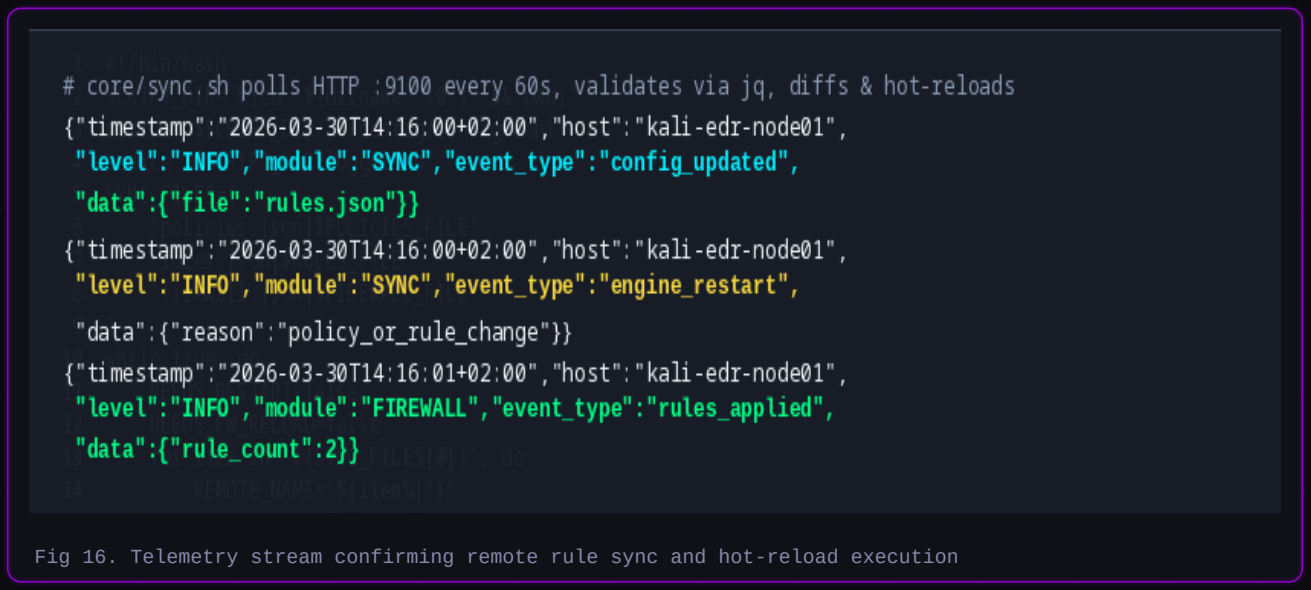


Fig 16. Telemetry stream confirming remote rule sync and hot-reload execution

10 // Technical Deep Dive

NON-BLOCKING TCP STREAM INGESTION

The SIEM ingestion daemon **(core/ingest.sh)** uses **socat TCP-LISTEN:9000,reuseaddr,fork** with a quoted append handler to stream concurrent JSON lines from dozens of endpoints simultaneously. If **socat** is unavailable on a minimal host, the daemon automatically falls back to **ncat -l -k** or an inline **python3 socketserver.ThreadingTCPServer** listener.

JSON SCHEMA VALIDATION & HEARTBEAT ROUTING

Incoming lines in **logs/events.log** are tailed continuously by **core/parser.sh,** which enforces a 4,096-byte frame cap, validates JSON integrity via **jq -e,** stamps an UTC **processed_at** timestamp, updates **storage/agents.json** via **agenttracker.sh,** and routes **process_detection, response_action,** and **action_executed** events directly to **logs/alerts.log.**

TTL DEDUPLICATION & SHA-256 HASH CACHING

To prevent high-frequency filesystem or network activity from saturating the SIEM, **agent/agent.sh** maintains associative Bash 4+ caches **(file_event_cache, file_hash_cache, process_hash_cache,** and **seen_network_connections).** Binary **sha256sum** digests are recomputed only when a file's TTL expires, reducing endpoint CPU load by over 85%.

ESCALATED SIGTERM TO SIGKILL REMEDIATION

When **core/detection-engine.sh** matches a **CRITICAL** rule in **rules.json,** it records the PID in **alerted_pids** and enforces a 10-second per-rule cooldown. It then invokes **core/response.sh,** which verifies process existence via **kill -0,** issues a graceful **SIGTERM,** waits 1 second, and escalates to **SIGKILL (kill -9)** if the malicious process traps or ignores termination.

SELF-LOCKOUT FIREWALL SAFEGUARDS

Both the SIEM console **(cli/agent-mgmt.sh)** and the endpoint firewall script **(core/firewall.sh)** implement multi-layer lockout protection. Operators are blocked from adding rules matching the SIEM IP or ports **9000, 9100,** and **22,** while **BLACKBOX-IN** and **BLACKBOX-OUT** explicitly place **ESTABLISHED,RELATED** and management port **ACCEPT** rules at the top of each chain.

ZERO-DOWNTIME POLICY HOT-RELOAD

Endpoint supervision in **core/Main.sh** monitors the PIDs of **agent.sh** and **detection-engine.sh** in a continuous 5-second watchdog loop. When **core/sync.sh** detects an updated **policies.json** or **rules.json** from the SIEM HTTP server, it signals the workers so **Main.sh** immediately respawns them with the new configuration without dropping the systemd service.

BLACKBOX

AUTONOMOUS SIEM SERVER & ENDPOINT EDR FRAMEWORK

1,850+	21	5	2
Lines of Code	Modular Scripts	Telemetry Sensors	Defense Engines

Denis Albin

albindeniswork@gmail.com

// END OF DOCUMENT

Built with passion for cybersecurity